

Práctica 6: Refactorizaciones

Memoria de la práctica

Juan Francisco González

Juan Calle

1. Introducción

En esta práctica se ha trabajado sobre una aplicación de gestión de transportes entregada como código heredado. El objetivo no era añadir funcionalidad nueva, sino estudiar el sistema original, medir su calidad interna y aplicar una refactorización que mejorase el diseño sin cambiar el comportamiento observable del programa.

Tal y como se pedía en el enunciado, se han preparado dos proyectos distintos: uno con el sistema original y otro con la versión refactorizada. Esto permite comparar de forma clara el antes y el después, tanto a nivel de estructura como de métricas de calidad.

En esta memoria se recogen los puntos importantes de la entrega: de dónde salen las métricas, qué cambios se han hecho, cómo quedan los diagramas de clases, qué clases contribuyen al **CBO** de cada una y, sobre todo, si las métricas reflejan de verdad la mejora conseguida después de refactorizar.

2. Criterio seguido para calcular las métricas

Para calcular las métricas se ha seguido el criterio explicado en la teoría proporcionada con la práctica, en concreto el tema 6.1. En ese material aparecen las definiciones de **WMC**, **WMCn**, **CBO**, **DIT** y **NOC**, además de la forma de calcular la complejidad ciclomática y la complejidad cognitiva en Java.

El criterio aplicado ha sido el siguiente. **WMC** se obtiene sumando la complejidad ciclomática de los métodos y constructores definidos en la clase. **WMCn** es ese valor normalizado por el número de métodos considerados. **CCog** se ha calculado sumando la complejidad cognitiva de los métodos, y **CCogn** se ha obtenido también de forma normalizada.

En el caso del **CBO** no se ha puesto solo el número, sino también de dónde sale. Es decir, en las tablas se indica qué clases contribuyen a ese acoplamiento. Por ejemplo, si una clase A usa directamente las clases C y D, entonces C y D contribuyen al CBO de A. En esta práctica también se han tenido en cuenta las excepciones y las clases de biblioteca que aparecen referenciadas directamente en el código, porque el propio enunciado lo recalca.

Además, en ambos proyectos se ha dejado comentado dentro del código cómo se obtiene el **WMC** y la **CCog** de cada clase, tal y como se pedía expresamente.

3. Situación inicial del sistema

La aplicación original permite registrar conductores, almacenar transportes y calcular el sueldo de cada conductor a partir de un sueldo base y de varios complementos. El problema no está en la funcionalidad, que es sencilla, sino en cómo está repartida esa lógica dentro del código.

Los dos puntos más problemáticos del diseño original son la clase Conductor, que asume demasiada lógica de negocio en el método sueldo(), y la clase GestionTransportesGUI, cuyo main() concentra menú, lectura de datos, creación de objetos y parte de la lógica de aplicación. Eso hace que el sistema sea más difícil de entender y de modificar.

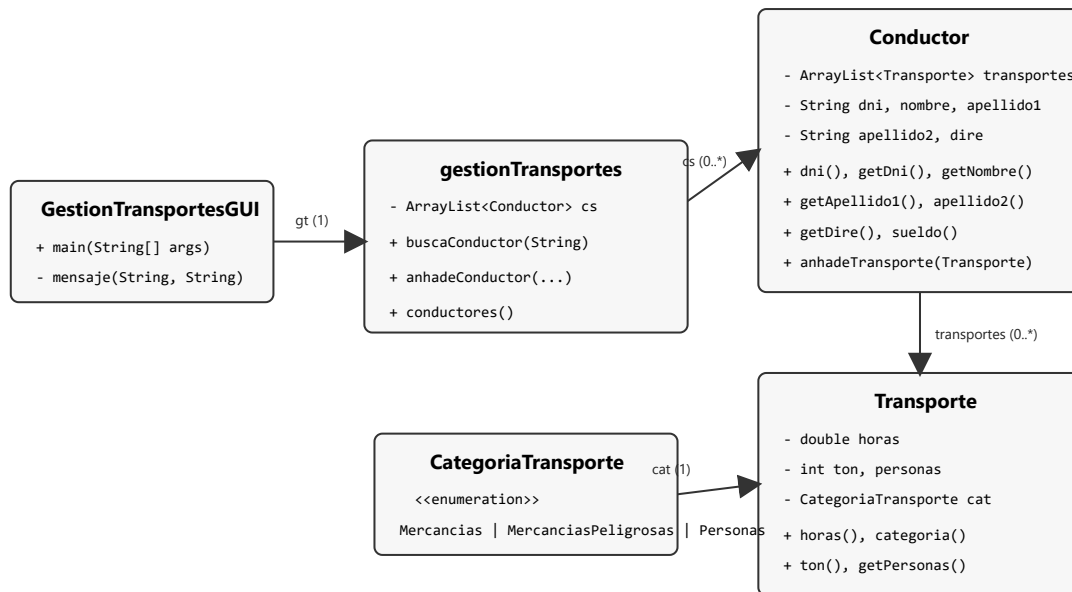


Figura 1. Diagrama de clases del sistema original.

3.1 Métricas del sistema original

Clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
CategoriaTransporte	0	0.00	0	0.00	3	0	0
Transporte	9	1.80	3	0.60	4	0	0
Conductor	18	2.00	9	1.00	7	0	0
gestionTransportes	6	2.00	4	1.33	5	0	0
GestionTransportesGUI	18	9.00	34	17.00	10	0	0

3.2 Clases que contribuyen al CBO en el sistema original

Clase	Clases que contribuyen a su CBO
CategoriaTransporte	Conductor, Transporte, GestionTransportesGUI

Clase	Clases que contribuyen a su CBO
Transporte	CategoriaTransporte, Conductor, GestionTransportesGUI, IllegalArgumentException
Conductor	ArrayList, String, Transporte, CategoriaTransporte, IllegalArgumentException, gestionTransportes, GestionTransportesGUI
gestionTransportes	ArrayList, List, String, Conductor, GestionTransportesGUI
GestionTransportesGUI	LinkedList, List, Lectura, Menu, Mensaje, String, Conductor, gestionTransportes, Transporte, CategoriaTransporte

Viendo estos valores, la clase más problemática es `GestionTransportesGUI`, porque concentra el mayor valor de complejidad total y también la mayor complejidad normalizada. `Conductor` también destaca porque contiene una lógica que realmente debería estar más cerca del propio concepto de transporte.

4. Refactorización realizada

La refactorización se ha hecho en un segundo proyecto Maven independiente, tal y como pedía la práctica. La idea no ha sido rehacer el programa desde cero, sino mejorar el diseño manteniendo el comportamiento del sistema y comprobando después que todo seguía funcionando.

Los cambios principales han sido los siguientes:

- Se han creado dos proyectos separados, uno original y otro refactorizado, para poder comparar ambos estados del sistema.
- Se ha sustituido el código de tipo basado en `CategoriaTransporte` por una jerarquía de clases con `Transporte`, `TransportePersonas`, `TransporteMercancias` y `TransporteMercanciasPeligrosas`.
- Se ha reemplazado la lógica condicional del cálculo del sueldo por polimorfismo, moviendo ese comportamiento a los propios transportes.
- Se ha simplificado la clase `Conductor`, que ahora se limita a recorrer sus transportes y sumar el importe devuelto por cada uno.
- La clase de gestión se ha regularizado y se ha pasado de una lista lineal a una estructura `Map` indexada por DNI.
- Se han corregido nombres y responsabilidades para que el código sea más coherente y más fácil de leer.
- Se han ampliado las pruebas para cubrir la nueva estructura del dominio.

El cambio más importante ha sido sacar de `Conductor` la responsabilidad de decidir cuánto cobra cada tipo de transporte. En la versión original el conductor tenía que preguntar por la categoría y aplicar reglas diferentes con condicionales. En la versión refactorizada cada subtipo de transporte conoce su propio cálculo, que es justo lo que se espera en un diseño más orientado a objetos.

5. Situación final del sistema refactorizado

Después de la refactorización, el sistema mantiene la misma idea general, pero con un reparto de responsabilidades mucho más claro. La lógica del cálculo salarial ya no está concentrada en

Conductor, sino distribuida entre clases pequeñas y especializadas.

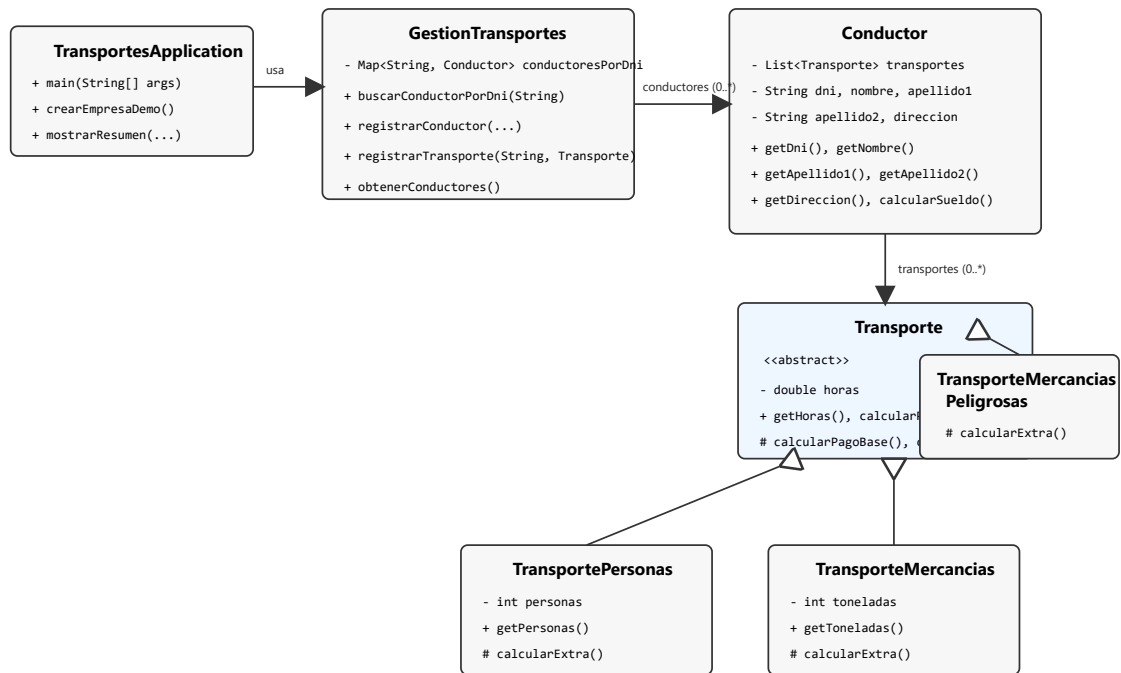


Figura 2. Diagrama de clases del sistema refactorizado.

5.1 Métricas del sistema refactorizado

Clase	WMC	WMCn	CCog	CCogn	CBO	DIT	NOC
Conductor	9	1.13	1	0.13	7	0	0
GestionTransportes	7	1.40	2	0.40	8	0	0
Transporte	5	1.25	1	0.25	5	0	2
TransportePersonas	5	1.67	2	0.67	3	1	0
TransporteMercancias	4	1.33	1	0.33	4	1	1
TransporteMercanciasPe ligrosas	2	1.00	0	0.00	2	2	0
TransportesApplication	7	1.40	2	0.40	6	0	0

5.2 Clases que contribuyen al CBO en el sistema refactorizado

Clase	Clases que contribuyen a su CBO
Conductor	List, ArrayList, Objects, String, Transporte, GestionTransportes, TransportesApplication

Clase	Clases que contribuyen a su CBO
GestionTransportes	Map, LinkedHashMap, List, String, Conductor, Transporte, IllegalArgumentException, TransportesApplication
Transporte	IllegalArgumentException, Conductor, GestionTransportes, TransportePersonas, TransporteMercancias
TransportePersonas	Transporte, IllegalArgumentException, TransportesApplication
TransporteMercancias	Transporte, IllegalArgumentException, TransporteMercanciasPeligrosas, TransportesApplication
TransporteMercanciasPeligrosas	TransporteMercancias, TransportesApplication
TransportesApplication	System, GestionTransportes, Conductor, TransportePersonas, TransporteMercancias, TransporteMercanciasPeligrosas

6. Análisis y razonamiento sobre las métricas

Esta es la parte más importante de la práctica, porque no basta con enseñar tablas. Hay que justificar si los cambios en las métricas reflejan realmente una mejora del código o si solo cambian los números porque la estructura del sistema ahora es distinta.

6.1 Casos en los que la mejora se ve claramente

Conductor

Es el caso más claro de toda la práctica. El **WMC** baja de 18 a 9 y la **CCog** pasa de 9 a 1. Esto encaja perfectamente con el cambio realizado: el conductor deja de decidir cuánto cobra cada transporte y se limita a sumar importes. Aquí sí se puede decir que la métrica refleja de forma muy directa la mejora del diseño.

GestionTransportesGUI frente a TransportesApplication

En el sistema original el punto de entrada del programa concentraba demasiada lógica y demasiadas dependencias. En la versión final se sustituye por una clase más simple, con menos complejidad y una responsabilidad más acotada. También en este caso los números acompañan bastante bien a la mejora observada en el código.

6.2 Casos que hay que interpretar con más cuidado

GestionTransportes

Esta clase es la que mejor demuestra que una métrica no debe leerse sola. Su **WMC** bruto sube ligeramente de 6 a 7 y el **CBO** también pasa de 5 a 8. Si se mirase solo eso, podría parecer que la clase empeora.

Sin embargo, la lectura completa es distinta. La clase participa ahora de una forma más clara en el diseño y coordina más tipos del dominio, por eso su CBO crece. Aun así, su **CCog** baja de 4 a 2 y su

WMCn también mejora. Es decir, la clase se relaciona con más elementos, pero sus métodos son más sencillos y más mantenibles.

DIT y NOC

En el sistema original estas métricas valían cero porque no existía una jerarquía de clases. En la versión refactorizada aparecen valores mayores porque se ha introducido herencia y polimorfismo. Eso no significa que el diseño haya empeorado, sino que la solución elegida ya no se basa en condicionales sino en especialización.

Por tanto, en esta práctica **DIT** y **NOC** no deben interpretarse como un empeoramiento automático, sino como una consecuencia lógica del cambio de diseño.

6.3 Comparación global

Indicador global	Original	Refactorizado
Número de clases medidas	5	7
WMC medio	10.20	5.57
WMC máximo	18	9
WMCn medio	2.96	1.31
CCog medio	10.00	1.29
CCog máximo	34	2
CCogn medio	3.99	0.31
CBO medio	5.80	5.00
CBO máximo	10	8
DIT máximo	0	2
NOC máximo	0	2

La comparación global es positiva. Aunque el número de clases aumenta, la complejidad queda mucho mejor repartida. Bajan claramente los valores medios y máximos relacionados con complejidad, que era precisamente donde estaba el principal problema del diseño original.

6.4 Conclusión razonada sobre si las métricas reflejan la mejora

La conclusión es que **sí**, las métricas reflejan bastante bien la mejora introducida, pero no todas lo hacen de la misma manera. Las métricas de complejidad (**WMC**, **WMCn**, **CCog** y **CCogn**) son las que mejor captan el efecto de la refactorización, porque muestran con claridad que los métodos problemáticos se han simplificado.

En cambio, métricas como **CBO**, **DIT** y **NOC** hay que leerlas con más contexto. Al introducir una jerarquía de clases y usar polimorfismo es normal que aparezcan nuevas relaciones entre clases. Eso

puede hacer que algunos valores suban, pero no por ello el diseño es peor. De hecho, en este caso ese aumento es coherente con una solución mejor orientada a objetos.

Por tanto, la mejora del sistema sí se ve en las métricas, pero la interpretación correcta sale de combinarlas con el análisis del diseño. Precisamente por eso no basta con poner las tablas: hay que razonar qué significan esos cambios y si encajan o no con la refactorización realizada.

7. Pruebas y comprobación final

Para comprobar que la refactorización no introducía errores se ejecutaron pruebas en ambos proyectos. En el proyecto original se ejecutó `mvn test` con resultado correcto. En el proyecto refactorizado se ejecutó `mvn verify`, obteniendo todas las pruebas en verde y generando además el jar ejecutable final.

En la versión refactorizada no solo se mantuvieron las pruebas existentes, sino que se añadieron pruebas nuevas sobre `GestionTransportes` y sobre cada subtipo de transporte, para que el nuevo diseño quedase cubierto de una forma más coherente.

La entrega final incluye:

- el proyecto original completo;
- el proyecto refactorizado completo;
- los diagramas inicial y final;
- el jar ejecutable del proyecto refactorizado;
- esta memoria final en PDF.

8. Conclusión

En esta práctica se ha visto que refactorizar no consiste solo en cambiar nombres o dejar el código más ordenado, sino en repartir mejor las responsabilidades para que el diseño represente mejor el problema. En este caso, el cambio clave ha sido sustituir condicionales por polimorfismo y sacar de `Conductor` una lógica que no le correspondía.

El resultado final es un sistema más claro, más fácil de ampliar y con una complejidad mucho mejor distribuida. Las métricas apoyan esta conclusión, especialmente las de complejidad, y el análisis del **CBO**, **DIT** y **NOC** permite justificar por qué algunos valores deben interpretarse con más cuidado.

Nota final: los cálculos detallados de WMC y complejidad cognitiva se han dejado comentados en el código fuente de ambos proyectos, tal y como pedía el enunciado.